The Skynet Node Operational Grimoire

Distributed Spatial Architectures & Telemetry Control Planes

Lead Systems Administrator
devsecfranklin@bitsmasher.net

May 2026

Contents

1

Distributed Node Topology

The bitsmasher.net infrastructure is established on a decentralized, heterogeneous compute grid. Node assignment splits heavy I/O operations, JVM world-state execution, and on-device neural inferencing across optimized platforms.

1.1 Core Compute Grid Nodes

Chonk The primary gameplay execution engine. It hosts the Paper server and coordinates low-level coordinate mutations via custom bridge interfaces.

Skynet The local AI orchestration node. It processes neural pipeline coordination, NBT file manipulation, and runs the local spatial engines.

Stargate The high-throughput Model Context Protocol (MCP) hub and network routing plane.

Edge-T The physical network boundary agent handling telemetry interception.

1.2 System Metrics & Runtime baselines

To ensure predictable execution timings under heavy block modifications, the runtime states must align with the parameters defined in Table 1.1.

Table 1.1: Distributed Cluster Runtime Configurations

Node	Platform / Engine	Vessel Target	Network Port	Purpose
Chonk	Paper 26.1.5	Java 25	25565 / 25575	World Persistence
Skynet	Python 3.11 / venv	Debian Bookworm (rpi)	–	T2BM Orchestration
Stargate	Go / Python Daemons	Linux AMD64	5005	MCP / Tel. Hub
Edge-T	LiteRT / Mendel OS	i.MX 8M ARM64	–	Boundary Vision

2

Hardware Acceleration & Voxelization Mathematics

To eliminate high CPU thread wait times during procedural generation on Chonk, neural model execution is strictly offloaded to hardware edge accelerators.

2.1 Hardware Acceleration Framework

- **Vision Pipeline:** Quantized integer models run on the Edge-T node's Coral Edge TPU. The host relies on the Mendel Development Tool (MDT) and SSH key authentication secured by `mdt-keymaster`.
- **Spatial Mapping:** Spatial engines on Skynet communicate directly with the local Hailo-8L NPU. Models must compile to the Hailo Executable Format (`.hef`) to enable direct memory access pipelines.

2.2 Voxel Mutation Calculation

Voxel placement calculations utilize a 3D coordinate transformation pipeline to map abstract model outputs to specific world delta regions.

Let $P_{local} = (x, y, z)^T$ be the local coordinate inside a structured schematic origin. The target world coordinate vector P_{world} on Chonk is computed using a translation offset vector $T = (T_x, T_y, T_z)^T$ and a rotation matrix $R(\theta)$:

$$P_{world} = R(\theta) \cdot P_{local} + T \tag{2.1}$$

Where $R(\theta)$ for a rotation of angle θ around the vertical Y-axis is defined by:

$$R_y(\theta) = \begin{pmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{pmatrix} \tag{2.2}$$

To avoid chunk desynchronization or locking during high block change throughput events, the processing latency (L) on the NPU must satisfy the performance inequality:

$$L_{npu} + \delta_{rcon} < \frac{1}{\text{TPS}} \quad \text{where TPS} \geq 20.0 \tag{2.3}$$

3

Schematic & Injection Pipeline

The Text-to-Building (T2BM) generation pipeline converts spatial parameters into physical block data arrays inside the active world space.

3.1 Deployment Safety Checks

To protect the integrity of human-built architecture and designated cluster facilities, the system executes an automated test framework prior to dispatching any RCON payload:

```
# Navigate to repository root
cd /home/skynet/game-skynet-minecraft

# Initialize and execute pipeline validation suite
PYTHONPATH=./src ./venv/bin/python3 -m pytest \
    test/test_schematic_metadata_integrity.py \
    test/test_schematic_boundary_safety.py
```

Listing 3.1: Pre-deployment Verification Command Line

3.2 Validation Vectors

1. **Metadata Integrity Check:** Compares schematic headers against authorized NBT coordinate profiles to confirm structural bounds.
2. **Boundary Safety Check:** Intercepts injection payloads if coordinates intersect with restricted zones, such as the neural operation field or core transport lines.
3. **Payload Transit Auditing:** Tails `logs/test`$_{c}$`ore.log`and`logs/hailort.log`$to verify that payload structu$

3.3 Git LFS & World Synchronization

Binary files (`.schem`, `.mca`, and `.dat`) are handled strictly via Git LFS. Before performing commits, the world state must be frozen to prevent backing up partially written or corrupted regions:

```
# Freeze chunk flushing and lock auto-save routines on the live server
./rcon.sh "save-off"
./rcon.sh "save-all flush"

# Execute LFS synchronization
git add world/region/*.mca
git commit -m "sysadmin: world region state snapshot"
git push origin master
```

```

# Restore server-side auto-save routines
./rcon.sh "save-on"
```

Listing 3.2: World Freeze and Sync Protocol

4

Operational Protocols & Telemetry

Our distributed nodes run asynchronous telemetry listeners to monitor server status and coordinate transport actions in real time.

4.1 Socket Configuration and Ports

Communication channels strictly adhere to the internal network matrix documented in Table 4.1.

Table 4.1: System Port Interface Matrix

Port	Protocol	Traffic Type	Service Endpoint
25565	TCP	Game Client Handshakes	Paper Minecraft Server
25575	TCP	Administrative Console	RCON Control Bridge
5005	TCP	Telemetry Events	CART_PASS Async Listener

4.2 Telemetry Event Processing

When a physical minecart triggers an accelerated detector rail array on Chonk, the telemetry sender encapsulates metadata regarding the transit and forwards it to Stargate on Port 5005.

Let E_{cart} be the telemetry event package. The serialized JSON envelope must strictly map to the following payload schema:

```
{
  "event": "CART_PASS",
  "node": "CHONK-01",
  "data": {
    "rail_id": "WASHINGTON_TERM_04",
    "coordinates": {
      "x": -1042,
      "y": 64,
      "z": 1289
    },
    "timestamp": 1778346300
  }
}
```

Listing 4.1: CART_PASS Telemetry JSON Schema

4.3 Debugging Pipelines

If an AI agent confirms building generation but changes are not reflected in the live world, administrators must isolate the pipeline drop points:

$$\text{T2BM Output} \xrightarrow{\text{Validated}} \texttt{test\_core.log} \xrightarrow{\text{Transited}} \texttt{hailort.log} \xrightarrow{\text{Injected}} \text{Chonk RCON} \tag{4.1}$$

By sequentially running the unit test targets via `pytest`, drop points can be identified and cleared before pipeline timeouts occur.